

Laboratorio No. 1 - Adquisición y Conversión ADC con Arduino (ADC: 0-5V), Monitor Serial, y Graficador MATLAB (GUI), con Red de Resistencias

Santiago Cadena, Maryuri Medina, Layla Rasch
{scadena, lrasch, mmedinaro}@unal.edu.co
Ingeniería electrónica. Laboratorio de Instrumentación y Medidas.

Resumen

A lo largo del presente informe se dan a conocer las diferentes mediciones de voltaje para una red de resistencias en serie tomadas por medio de multímetro, el serial monitor de un Arduino y por medio de la interfaz GUI de Matlab con el objetivo de comparar cada uno de los métodos de presentación de datos con el valor medido.

Index Terms

Medición, Presentación de datos, Interfaz.

I. INTRODUCCIÓN

A día de hoy, es esencial recoger y registrar datos durante un experimento para monitorear y evaluar los distintos circuitos y sistemas diseñados para propósitos específicos. Sin embargo, a menudo se subestima la importancia de cómo se presentan estos datos y los resultados obtenidos tras la implementación de estos circuitos y sistemas para un desarrollo o situación particular. Así, se ha avanzado significativamente en la presentación de datos de manera que resulte atractiva para los clientes o usuarios que interactúan con las interfaces de usuario creadas para facilitar la revisión y visualización de datos de manera amena y cómoda.

Igualmente, las metodologías para la recogida y análisis de datos, así como los tipos de señales utilizadas, han evolucionado. Hace décadas predominaban las señales analógicas, pero actualmente se observa una mezcla de señales analógicas y digitales, adaptándose al ámbito de aplicación. Este incremento en el uso de señales digitales ha llevado al desarrollo de diversos equipos y dispositivos, como Arduino, un microcontrolador de plataforma de hardware de código abierto que facilita el manejo de señales analógicas y digitales gracias a su convertidor de analógico a digital (ADC) integrado.

El surgimiento de tecnologías que permiten el uso simultáneo de señales analógicas y digitales o su conversión de forma eficiente ha promovido el desarrollo de software especializado, como Matlab, el cual es un software de alto rendimiento que ofrece capacidades para el manejo de problemas, simulación, modelado y visualización de datos. Este informe se enfoca en cómo Matlab se utiliza como una herramienta para la visualización y procesamiento de datos, permitiendo la creación de diversas representaciones gráficas.

II. OBJETIVOS

- Probar la adquisición y conversión ADC con el Arduino, utilizando 8 resistencias.
- Probar la presentación de datos y su registro usando la herramienta-software IDE de Arduino.
- Investigar y Crear una interfaz GUI usando Matlab, para hacer lectura y registro del ADC de ARDUINO, en tiempo real.

III. MARCO TEÓRICO

III-A. Arduino

Arduino es una plataforma de desarrollo de prototipos electrónicos que se basa en una placa de circuito impreso (PCB) con un microcontrolador programable. La placa Arduino es un dispositivo que se puede conectar a un ordenador mediante un cable USB y que permite interactuar con el entorno a través de sensores y actuadores. La placa Arduino está diseñada para ser fácil de usar y versátil, lo que la convierte en una herramienta popular tanto para principiantes como para expertos en electrónica.

La placa Arduino es un microcontrolador programable que se basa en el procesador ATmega, que es un microcontrolador de 8 bits. La placa Arduino tiene una memoria de 2 KB para el programa principal y 128 bytes para la memoria de datos. Además, tiene un puerto USB, un puerto serial, 14 entradas digitales y 6 entradas analógicas.

Además de la placa, la comunidad de Arduino ha desarrollado una serie de librerías y herramientas de software que facilitan el desarrollo de proyectos. Estas herramientas incluyen: Arduino IDE, este es un entorno de desarrollo integrado (IDE) que permite programar la placa Arduino y compilar el código fuente; librerías, una colección de código reutilizable que permite a los usuarios agregar funcionalidades específicas a sus proyectos; hardware, la comunidad ha desarrollado una serie de extensiones y accesorios que permiten ampliar las capacidades de la placa principal.

III-B. Matlab

Se trata de un entorno de desarrollo y un lenguaje de programación diseñado específicamente para tareas de cálculo numérico y simulación. Posee características clave que lo hacen una herramienta eficaz para áreas como ingeniería, ciencias naturales e investigación. Matlab facilita la realización de cálculos y experimentos de forma interactiva, lo cual mejora la depuración y el análisis de datos. También ofrece funcionalidades avanzadas de visualización y creación de gráficos para una representación efectiva de datos y resultados. Es ampliamente utilizado en el análisis de señales, procesamiento de datos, simulaciones numéricas y modelado matemático.

Incluye Simulink, una herramienta para el modelado y simulación de sistemas dinámicos, aplicable en control, electrónica y sistemas.

Además, MATLAB se destaca por su versatilidad, ya que se utiliza en áreas como aprendizaje automático, procesamiento de señales, procesamiento de imágenes, visión artificial, comunicaciones y finanzas computacionales.

III-C. Interfaces gráficas de Usuario

Una interfaz gráfica de usuario (GUI por sus siglas en inglés) es un tipo de interfaz que permite a los usuarios interactuar con dispositivos electrónicos a través de elementos visuales como botones, menús desplegables, ventanas y otros componentes gráficos. Estas interfaces facilitan la interacción entre el usuario y el sistema informático al presentar la información de manera visual e intuitiva, lo que simplifica la navegación y el uso de programas o dispositivos.

Las interfaces gráficas de usuario son comunes en aplicaciones informáticas, dispositivos móviles, electrodomésticos inteligentes y sistemas embebidos. Su diseño busca hacer que la interacción con la tecnología sea más accesible para usuarios de diferentes niveles de experiencia, permitiendo realizar acciones complejas de forma sencilla a través de una representación visual amigable.

IV. DESARROLLO

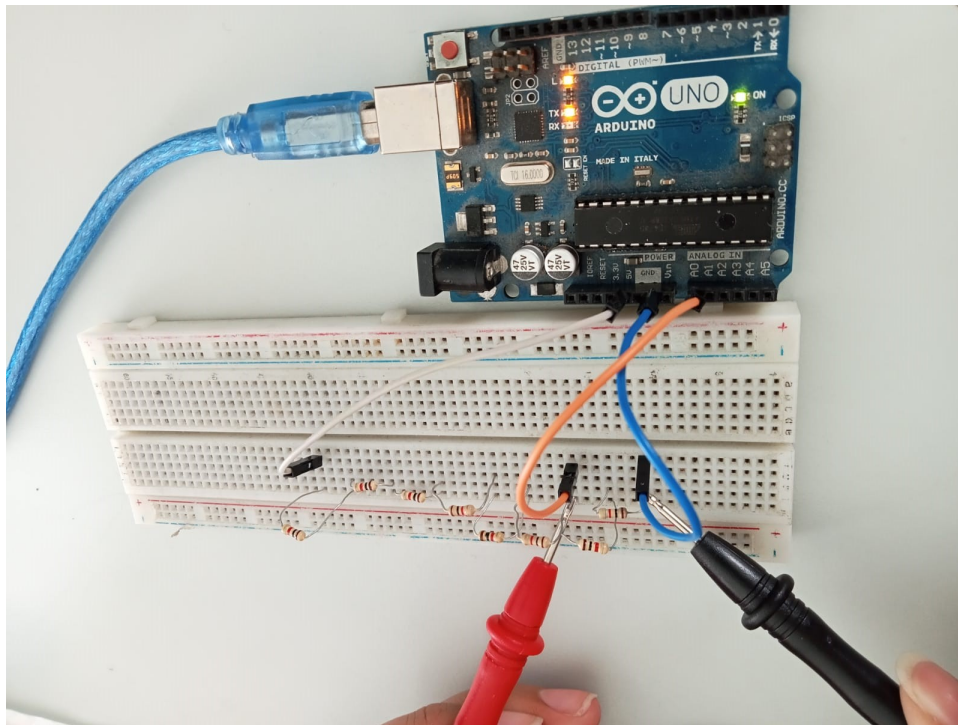


Figura 1. Esquema de red 8 resistencias empleado para el desarrollo de la práctica

Para realizar las mediciones de las caídas de potencial con los diferentes medios previstos, empleamos una red compuesta por 8 resistencias conectadas en serie a una fuente de 5V. De esta manera, cada resistencia recibe una tensión de $0,625V$,

considerando la distribución uniforme de voltaje. En este contexto, nuestro objetivo es evaluar con tres cifras significativas las características de cada medio y determinar si se presentan diferencias sutiles entre ellos.

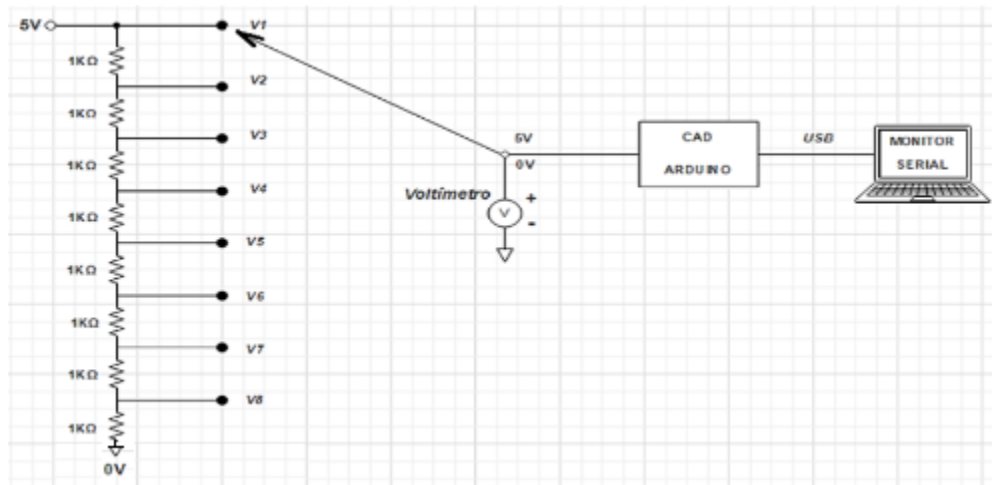


Figura 2. Esquema de red 8 resistencias para prueba de ADC Arduino. s

IV-A. Convertidor ADC de Arduino rama de 8 resistencias de $1k\Omega$

Para hacer la prueba del convertidor ADC de Arduino se ejecuta el siguiente código:

```

1  int lecture = A0;    // Pin para lectura analógica
2  float sensorValue = 0; // Variable que almacena el valor de voltaje de la medición
3
4
5  void setup() {
6      // Declaración como pin de entrada
7      pinMode(lecture, INPUT);
8      Serial.begin(9600);
9  }
10
11 void loop() {
12     // read the value from the sensor:
13     sensorValue = analogRead(lecture)*5.0/1023.0 ;
14     Serial.println(sensorValue);
15     delay(1000);
16 }

```

Listing 1. Código Arduino

Como la resolución del ADC en arduino UNO es de 10 bits, entonces representa un valor analógico de tensión entre 0 y $2^{10} - 1 (1023)$, siendo 1023 5V. La conversión se realiza con base en la siguiente ecuación:

$$Value = \frac{V_{lecture} \cdot 5}{1023}$$

Ahora, para mostrar los datos, pero sin registrarlos se hizo uso del Serial Plotter que viene integrado en Arduino.

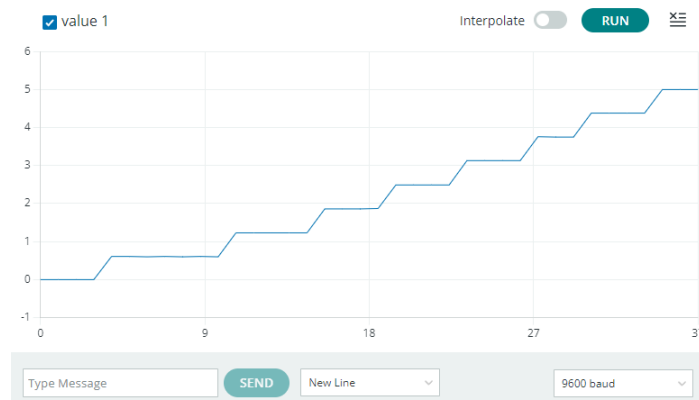


Figura 3. Serial Plotter Arduino

Para registrar los datos y compararlos con los datos arrojados del multímetro fue necesario hacer uso de un add-in en EXCEL llamado "Microsoft Data Streamer" para que tomara los datos recibidos por el puerto COM.

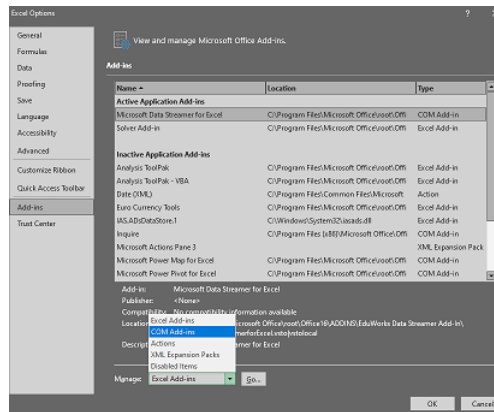


Figura 4. Add - in en EXCEL para leer datos de Arduino

Los datos registrados se muestran en la siguiente tabla:

Cuadro I
COMPARATIVA ENTRE MEDICIONES

# Resistencia	Voltaje(V)		
	Multímetro	Arduino	Real
0	0	0	0
1	0,625	0,616	0,625
2	1,249	1,237	1,25
3	1,866	1,862	1,875
4	2,501	2,488	2,5
5	3,108	3,123	3,125
6	3,744	3,749	3,75
7	4,361	4,379	4,375
8	4,995	5	5

Como se evidencia, no se observaron diferencias notorias en los datos del multímetro o arduino respecto al valor real teórico (sin considerar las tolerancias de las resistencias). Por esta razón, en las gráficas, se ve una casi solapada encima de la otra:

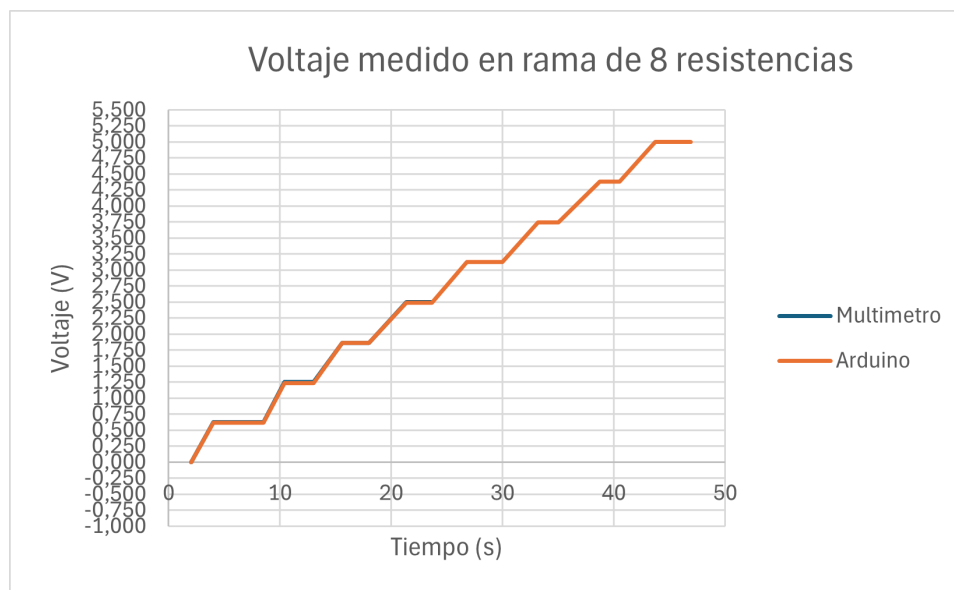


Figura 5. Comparación mediciones del Monitor Serial de Arduino y Multímetro

IV-B. GUI Matlab

Mediante el software de matlab, se pudo también visualizar cada uno de los datos a evaluar. Se instalaron los paquetes de Arduino, para conectar con la tarjeta; guide, un entorno de desarrollo para diseñar GUIs; y MATLAB Compiler SDK, un paquete para compilar aplicaciones de forma independiente.

Para el diseño, se procedió con el comando "GUI" a editar gráficamente. Se agregaron estos componentes: 3 botones, una gráfica y un label estático.

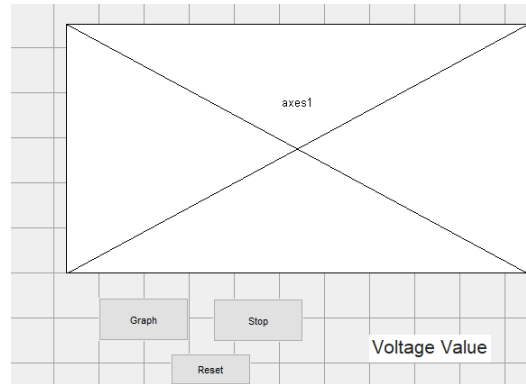


Figura 6. Diseño de GUI - Matlab

Los botones tuvieron asociados unos callbacks, en donde se escribió código dependiendo de las funcionalidades de cada uno (líneas 44,89,97), llamando a la variable global, una instancia de la clase Arduino (línea 32).

```

1
2
3 function varargout = GuideGrafica(varargin)
4
5
6 gui_Singleton = 1;
7 gui_State = struct('gui_Name',       mfilename, ...
8                   'gui_Singleton',   gui_Singleton, ...
9                   'gui_OpeningFcn',   @GuideGrafica_OpeningFcn, ...
10                  'gui_OutputFcn',    @GuideGrafica_OutputFcn, ...
11                  'gui_LayoutFcn',    [], ...
12                  'gui_Callback',     []);
13 if nargin && ischar(varargin{1})
14     gui_State.gui_Callback = str2func(varargin{1});
15 end
16
17 if nargout
18     [varargout{1:nargout}] = gui_mainfcn(gui_State, varargin{:});
19 else
20     gui_mainfcn(gui_State, varargin{:});
21 end
22
23
24 function GuideGrafica_OpeningFcn(hObject, eventdata, handles, varargin)
25
26 handles.output = hObject;
27
28 guidata(hObject, handles);
29 delete(instrfind({'Port'}, {'COM4'}));
30 clear a;
31
32 global a init_time x;
33 a = arduino('COM4');
34 init_time = 1;
35 x = 0;
36
37
38
39 function varargout = GuideGrafica_OutputFcn(hObject, eventdata, handles)
40
41 varargout{1} = handles.output;
42
43
44

```

```

45 function Graph_Callback(hObject, eventdata, handles)
46 set(handles.Stop,'UserData',0);
47 interv = 50; %intervalo hasta el que grafica en un cierto limite de x.
48
49 %set(handles.Stop,'UserData',0);
50 global x init_time a;
51
52 while(init_time<interv && handles.Stop.UserData==0)
53     voltageValue = readVoltage(a,'A0');
54     %Resistance = 9800 * (1 / ((5 / voltagevalue) - 1));
55     x = [x,voltageValue];
56     plot(x)
57     title('Voltage vs Time')
58     xlabel('Time(s)')
59     ylabel('Voltage(V)')
60     grid on
61     init_time = init_time+1;
62     ylim([0 6]);
63     pause(0.5);
64
65     voltageValue = readVoltage(a,'A0');
66     set(handles.VoltageValue,'String',voltageValue);
67 end
68
69
70 % Assuming this is the callback function for the "Start" button
71 function startButton_Callback(hObject, eventdata, handles)
72 % hObject    handle to startButton (see GCBO)
73 % eventdata  reserved - to be defined in a future version of MATLAB
74 % handles    structure with handles and user data (see GUIDATA)
75
76 % Start the loop for reading and plotting data
77
78
79 %}
80 function Edit_Callback(hObject, eventdata, handles)
81
82
83 function Edit_CreateFcn(hObject, eventdata, handles)
84 if ispc && isequal(get(hObject,'BackgroundColor'), get(0,'defaultUicontrolBackgroundColor'))
85     set(hObject,'BackgroundColor','white');
86 end
87
88
89 % --- Executes on button press in Stop.
90 function Stop_Callback(hObject, eventdata, handles)
91 handles.Stop.UserData=~handles.Stop.UserData;
92 % hObject    handle to Stop (see GCBO)
93 % eventdata  reserved - to be defined in a future version of MATLAB
94 % handles    structure with handles and user data (see GUIDATA)
95
96
97 % --- Executes on button press in Reset.
98 function Reset_Callback(hObject, eventdata, handles)
99 % hObject    handle to Reset (see GCBO)
100 % eventdata  reserved - to be defined in a future version of MATLAB
101 % handles    structure with handles and user data (see GUIDATA)
102 global x init_time
103 x=0;
104 init_time=1;

```

Listing 2. Código Diseño GUI MATLAB

Finalmente, al hacer las mediciones la GUI, quedó de la siguiente manera:

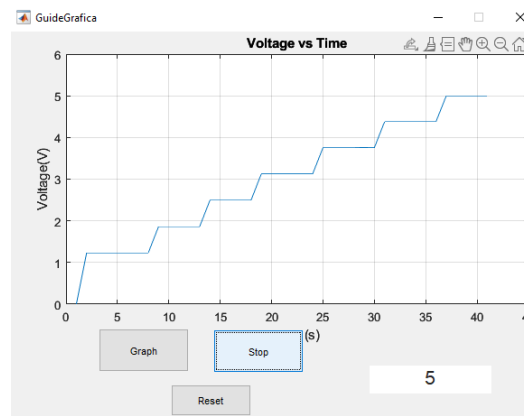


Figura 7. GUI de Matlab mostrando la medición de tensión en resistencias

V. ANÁLISIS DE RESULTADOS

A partir de los datos recogidos en la práctica y la experimentación a lo largo de la misma se puede afirmar que los métodos de medición son útiles para determinar el voltaje de diferentes mediciones de la red de 8 de resistencias, mas existen detalles que de deben tener en cuenta a la hora de determinar la exactitud y precisión de cada medida.

En primer lugar, para el multímetro digital se tiene por defecto según su hoja de datos un error de 0.5 %, el cual puede variar respecto a la temperatura o el ruido del ambiente, recogidos a partir de la alta impedancia de medición del voltaje ya que entre más alta sea es mas propensa a recoger ruido de ambiente. Esto es algo que si bien disminuye en el arduino al tener una impedancia menor, no garantiza que esta sea la mejor opción a la hora de tomar los datos.

Según distintas fuentes bibliográficas, la toma de voltaje dentro de este proceso de código del arduino presenta un error porcentual del 1.5 %, dado a partir de las impedancias internas y de la forma de medición realizada a partir de la división entre el valor medido multiplicado por el de referencia (5 V) y el rango de 1023. Ahora bien, a lo largo de la práctica se observó que la alimentación del arduino no siempre era 5V exactos, aumentando con ello error a la ecuación, la cual obvia la alimentación en 5V exactos.

Respecto a la toma de datos del GUI de Matlab se puede destacar que ofrece una herramienta versátil que permite obtener los valores de voltaje en tiempo real sin necesidad de atravesar la ecuación de toma de datos del arduino, a pesar de que las mediciones llegan por el mismo medio. Por lo cual se disminuye el error de calculo, a pesar de mantener el error proveniente de las impedancias internas.

Finalmente, al comparar las tres opciones, es importante considerar que todas ellas presentan pequeños errores que son inherentemente imposibles de eliminar por completo, como los errores humanos en la toma de mediciones o las impedancias de los cables y la protoboard, así como los posibles errores de las resistencias individuales (los cuales podrían reducirse mediante el uso de resistencias de precisión en lugar de comerciales). Sin embargo, al analizar detalladamente los valores recopilados durante la práctica, se observa que los datos son suficientemente similares entre sí con cualquiera de las tres opciones de medición, incluso coincidiendo prácticamente en sus cifras significativas, tanto en los datos del Arduino como en los del multímetro. Por lo tanto, no es posible tomar una decisión precisa basada únicamente en estos datos. En consecuencia, se optó por elegir la interfaz gráfica de usuario (GUI) de MATLAB como la mejor opción. Esto se debe a que permite extraer los datos de voltaje de manera más sencilla que con Arduino y, además, su versatilidad permite una configuración más amplia para compensar los errores mencionados anteriormente. De esta manera, se logra el objetivo principal de los laboratorios de la asignatura: obtener datos con el menor margen de error posible.

VI. CONCLUSIONES

- Los errores de laboratorio son una variable imposible de desaparecer por completo, por lo cual se debe tener en cuenta a la hora de los análisis de datos.
- La tarjeta Arduino presenta limitaciones respecto a su resolución de 10 bits, causando errores al pasar los datos analógicos tomados a bits. Sumado a esto el arduino el COM necesita de otro programa para leer los datos (en este caso excel), lo que dificulta el procedimiento en general.
- La herramienta de GUI de matlab es una poderosa opción para la creación de osciloscopios personalizados y aplicaciones de obtención de datos.

- El multímetro digital es una herramienta eficiente para cualquier práctica de laboratorio que cuente con medición de datos, es por ello que es conveniente tener uno siempre a disposición como método de comprobación de los datos.
- La herramienta de GUI de matlab es la mejor opción para la toma de datos de manera precisa, por lo que será la herramienta a emplear en el proyecto que se llevará a cabo a lo largo del semestre.

REFERENCIAS

- [1] H. Kopka and P. W. Daly, *A Guide to L^AT_EX*, 3rd ed. Harlow, England: Addison-Wesley, 1999.
- [2] magazine, “Arduino: ¿Qué es y para qué sirve?,” Blog de tecnología, Apr. 06, 2017. Available: <https://ibertronica.es/blog/productos/arduino/>. [Accessed: Mar. 04, 2024]
- [3] cgarcia, “¿Qué es Arduino y para qué sirve? — Escuela de programación, robótica y pensamiento computacional — Codelearn.es,” Codelearn.es, 2019. Available: <https://codelearn.es/blog/que-es-arduino-para-que-sirve/>. [Accessed: Mar. 04, 2024]
- [4] “¿Qué es MATLAB y dónde se usa?,” Avenuglobal.com, 2023. Available: <https://www.avenuglobal.com/noticias/que-es-y-para-que-sirve-matlab>. [Accessed: Mar. 04, 2024]
- [5] “¿Qué son las interfaces de usuario y cuáles son sus componentes,” KeepCoding Bootcamps, Jun. 09, 2022. Available: <https://keepcoding.io/blog/que-son-las-interfaces-de-usuario-componentes/>. [Accessed: Mar. 04, 2024]